Task 1: A Shared Shopping Cart

```
import Foundation
struct Product {
    let id: Int
    let name: String
    let price: Double
}

class CartStore {
    var quantities: [Int: Int] = [:]
    let products: [Product]

    init(products: [Product]) {
        self.products = products
    }

    func add(_ product: Product) {
        quantities[product.id, default: 0] += 1
    }

    func remove(_ product: Product) {
        guard let quantity = quantities[product.id], quantity > 0 else {
            return
        }
```

```
        quantities[product.id] = quantity - 1
    }

    func totalCount() -> Int {
        quantities.values.reduce(0, +)
    }

    func totalCost() -> Double {
        products.reduce(0) { total, product in
            total + product.price * Double(quantities[product.id, default: 0])
        }
    }
}

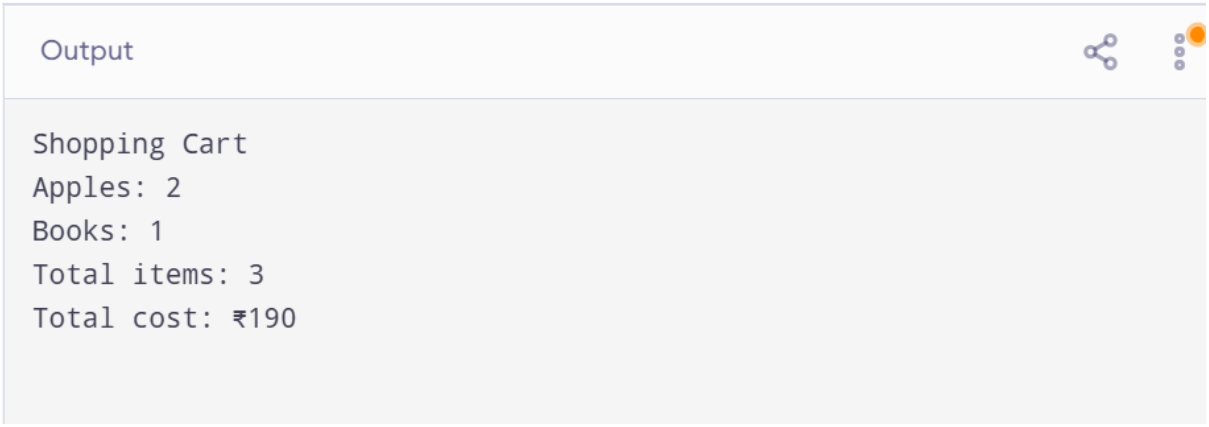
let apple = Product(id: 1, name: "Apple", price: 20)
let book = Product(id: 2, name: "Book", price: 150)

let cart = CartStore(products: [apple, book])

cart.add(apple)
cart.add(apple)
cart.add(book)

print("Shopping Cart")
print("Apples: \(cart.quantities[1, default: 0])")
print("Books: \(cart.quantities[2, default: 0])")
```

```
print("Total items: \(cart.totalCount())")
print("Total cost: ₹\(Int(cart.totalCost()))")
```



Task 2: Sign-up Form with Validation

```
import Foundation
func validateSignup(username: String, password: String, acceptedTerms: Bool) {
    if username.trimmingCharacters(in: .whitespaces).isEmpty {
        print("Enter a username")
    } else if password.count < 8 {
        print("Password must contain at least 8 characters")
    } else if !acceptedTerms {
        print("Please accept the terms")
    } else {
        print("Account created successfully")
    }
}

validateSignup(
```

```
    username: "Manoj",
    password: "swift1234",
    acceptedTerms: true
)

validateSignup(
    username: "Manoj",
    password: "abc",
    acceptedTerms: true
)

validateSignup(
    username: "Manoj",
    password: "swift1234",
    acceptedTerms: false
)
```

Output

```
Account created successfully
Password must contain at least 8 characters
Please accept the terms
```

Task 3: Debug Task — Four State Bugs

```
import Foundation
print("Bug 1: Counter resets whenever the view redraws.")
print("Fix: Store the counter using @State.")

print("\nBug 2: Screen does not update after tapping.")
print("Fix: Use @State for view-owned data.")

print("\nBug 3: Child changes do not update the parent.")
print("Fix: Pass a @Binding to the child view.")

print("\nBug 4: Modifying state during view rendering.")
print("Fix: Update state in an action, not inside the view body.")
```

Output

```
Bug 1: Counter resets whenever the view redraws.
Fix: Store the counter using @State.

Bug 2: Screen does not update after tapping.
Fix: Use @State for view-owned data.

Bug 3: Child changes do not update the parent.
Fix: Pass a @Binding to the child view.

Bug 4: Modifying state during view rendering.
Fix: Update state in an action, not inside the view body.
```